ADR-001: Multi-Agent Orchestration Pattern (Sequential Supervisor Pipeline)

Context:
EquipFlow requires a multi-step maintenance analysis workflow (Domain D5) that progresses from symptom identification to diagnostic/safety planning, and finally to work order generation. The ITI assessment mandates:

- ≥ 3 specialized agents + an orchestrator.
- Explicit typed contracts between agents (no free-form text).
- Mandatory controls: max-iteration breaker, per-step timeout, retry with backoff, and graceful degradation to plain RAG.
- Full observability (inspectable step-by-step by Run ID).

The *System Architecture Document* defines an "Agentic Coordination Layer" where a central Orchestrator routes intents through a specific sequence: `SymptomMatcher` $\to$ `Diagnostic & Safety Planner` $\to$ `Work Order Generator`.

Decision:
We will implement a **Sequential Supervisor Pipeline** pattern.

1. The Supervisor (Orchestrator)

A central coordination service residing in the Agentic Coordination Layer. It does not perform domain reasoning itself; instead, it:

- Receives the user intent and initializes an `AgentRun` context (with a unique Correlation ID).
- Enforces global constraints (Cost Governor pre-flight, global timeouts).
- Routes the payload sequentially to the three specialized agents.
- Catches agent failures and triggers **Graceful Degradation** (falling back to a standard RAG query without work order generation).

2. The Pipeline Steps (Specialized Agents)

Each agent is a distinct service with a restricted tool allow-list and explicit I/O contracts:

1. **SymptomMatcher**:
   - *Input*: `SymptomContext` (symptom text, equipment ID).
   - *Tools*: `SearchManuals` (RAG), `QueryFaultHistory`.
   - *Output*: `RankedFaults` (structured list of probable faults with confidence scores).
2. **Diagnostic & Safety Planner**:
   - *Input*: `RankedFaults`.
   - *Tools*: `GetEquipmentSpecs`, `GenerateSafetyChecklist`.
   - *Output*: `DiagnosticPlan` + `SafetyPrerequisites`.
3. **Work Order Generator**:
   - *Input*: `DiagnosticPlan`.
   - *Tools*: `ValidateBudget` (Cost Governor), `CreateWorkOrder` (Application Command).
   - *Output*: `WorkOrderDraftResult` (Work Order ID or refusal).

3. Mandatory Controls (ITI Compliance)

- **Typed Contracts**: Agents communicate via strict C# `record` types (e.g., `SymptomMatchResult`), preventing prompt-leakage and hallucination between steps.
- **Max-Iteration Breaker**: The Orchestrator limits internal agent tool-calling loops (ReAct loops) to a maximum of 3 iterations per agent.
- **Per-Step Timeout & Retry**: Each agent execution is wrapped in a 30-second timeout with a single retry using exponential backoff.
- **Graceful Degradation**: If the pipeline fails (e.g., LLM provider down, timeout, budget exhausted), the Orchestrator catches the exception and falls back to a plain RAG response, ensuring the user still gets a grounded answer.

Consequences:

- **Positive**: Strict adherence to Clean Architecture; agents are isolated and testable; deterministic fallback ensures system reliability; fully traceable via Correlation ID.
- **Negative**: Sequential execution introduces higher latency compared to parallel agent execution (mitigated by parallelizing tool calls *within* each agent).
- **Risks**: The Orchestrator is a single point of failure for the agentic workflow (mitigated by the graceful degradation to plain RAG).